

Lab 4: Single-Core and Multi-Core Systems

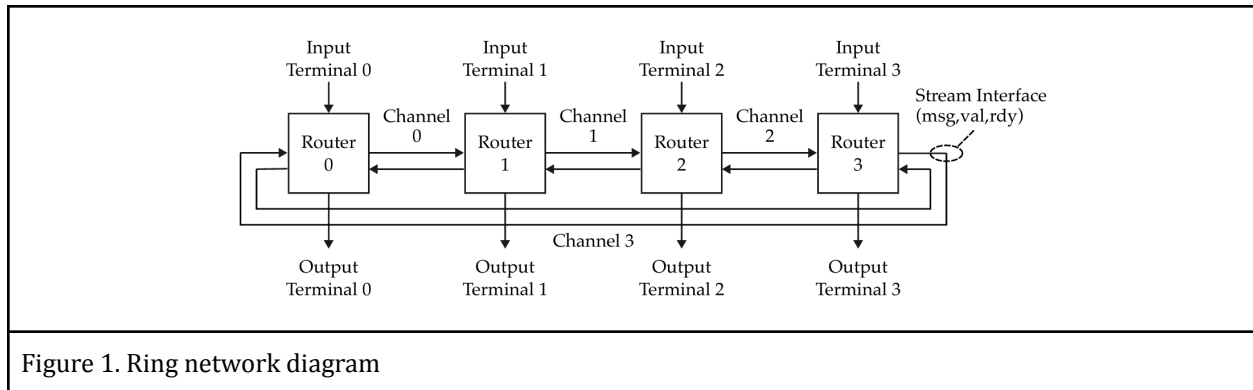
1. Introduction

Lab 4 extends upon the foundational concepts explored earlier in the semester to investigate the design and evaluation of both single-core and multi-core systems. For the baseline design, this lab integrates previously developed components—the iterative multiplier, pipelined processor, and FSM cache—into a complete single-core system with separate instruction and data caches. For the alternative design, this lab expands the single-core system into a multi-core system with four cores in a ring network used to interconnect processors to data caches and caches to main memory interfaces. The lab focuses on the incremental composition and testing of these systems, emphasizing the transition from single-threaded to multi-threaded applications. The key objective is to compare the performance of the single-core and multi-core systems, using various C programs as a benchmarking tool, while addressing trade-offs such as parallelism, coherence, and hardware complexity. This lab demonstrates the interaction between hardware and software design principles, leveraging modularity and test-driven development for effective system evaluation.

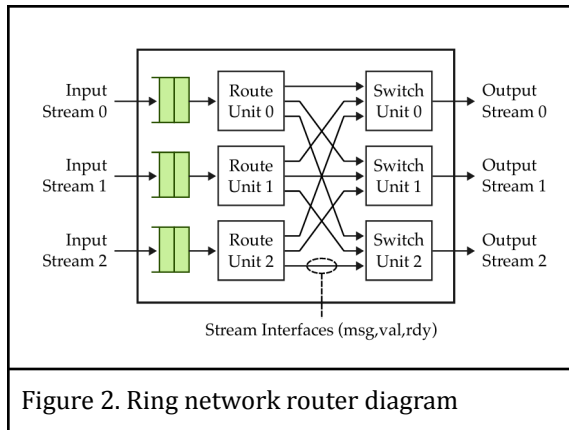
2. Alternative Design

The baseline design involves integrating a single-core system hardware and then developing a single-core system software to sort arrays of integers. It integrates a processor with separate instruction and data caches, ensuring modularity and simplicity in its architecture. However, the single-core design faces challenges with limited concurrency and potential bottlenecks in memory access due to its single-threaded nature. While the design efficiently supports basic computation and sorting tasks, its performance is constrained by the absence of parallel execution capabilities, limiting its ability to scale for larger workloads or exploit available data-level parallelism effectively. The single-core sorting algorithm implemented is a hybrid quicksort algorithm that uses insertion sort on smaller sized arrays due to its increased efficiency on small data.

The alternative design of the multi-core system addresses the limitations of the single-core system by introducing thread-level parallelism in a multi-core system, allowing for concurrent execution of programs. This design integrates four processors, each with private instruction caches, and a shared, banked data cache interconnected via ring networks. The ring network facilitates communication by routing memory requests and responses between cores, caches, and main memories. This allows for parallel processing, thus if one core is busy doing work, new work can be performed on a different core. The multi-core system software developed builds upon the single-core hybrid quicksort algorithm by splitting the array into four blocks (to be processed on each thread) and then merging the four sorted subarrays (again with multiple threads) to form a final sorted array.



The ring network is the key component of the alternative hardware design, consisting of four routers that handle data flow between input and output terminals. Network messages are injected into the ring network onto any of the four input terminals and then are eventually ejected on an output terminal. Each router has three input streams and three output streams, which correspond to messages from or to itself, the clockwise neighboring router, or the counterclockwise neighboring router. A visualization of this ring network is shown above in Figure 1.

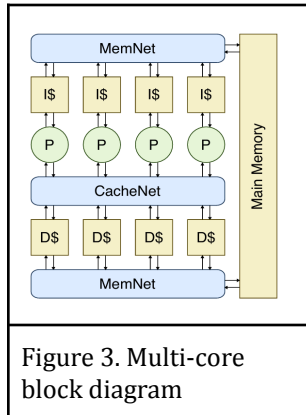


Each of the four cores contains a router, which serves as its central communication unit with other cores. There are three four-entry input queues, three route units, and three switch units contained within each router. Input queues in the router temporarily store incoming messages, ensuring orderly processing and preventing data loss during congestion. Route units determine the path taken by using a router destination and the current router id number to decide which switch unit to output to. Switch units determine the arbitration of routers, which determines which message can go out an output port. Figure 2 to the left illustrates the composition of each router unit.

For consistency, we assigned stream 0 to handle messages to and from the router itself, stream 1 for messages to and from the right (clockwise) neighboring router, and stream 2 for messages to and from the left (counterclockwise) neighboring router. In our implementation of the route unit, we adopted a shortest-path algorithm instead of the initial clockwise-only routing. If the destination router is the router itself, the message is routed to switch unit 0; If the destination router is the one unit to the right, the message is routed to switch unit 1; If the destination router is one unit to the left, it outputs to switch unit 2. In cases where the destination is two units away, either direction is equidistant; we arbitrarily choose the clockwise direction and route the message to switch unit 1.

After messages are output from the route units to the switch units, the switch units route them to the corresponding output stream of the router. Each switch unit contains arbitration logic that determines the sequence and order in which incoming messages from each stream are output. We implemented a fixed priority algorithm due to its simplicity. Input stream 1 is given the highest priority because it handles messages destined for routers two units away or one unit to the right; Input stream 2 is prioritized next, as it handles messages destined for routers one unit to the left; Finally, input stream 0 is given the lowest priority

since it deals with messages that have the most direct path, involving the router itself for both input and output.



Scaling out to the larger picture, developing the ring network allows the complete multi-core system to be put together. Note that the following integration code has already been provided in the lab code. The cache network connects four processors and four cache banks using request/response networks, with adapters converting between memory and network messages. The memory network links the caches to main memory, handling cache refills and evictions. These components, along with the shared, banked data cache, are integrated to form the multi-core data cache subsystem. Finally, the processors are connected to the cache network and data cache, creating a fully functional multi-core system capable of parallel processing with efficient memory access and inter-core communication. A block diagram of the overall multi-core architecture is displayed in Figure 3 to the right.

3. Testing Strategy

To implement this lab, our group adopted an incremental, test-driven approach to complete both the baseline and alternative designs. We thoroughly tested the hardware and software components as we progressed. For hardware testing, we utilized the Python-based testing framework that we had become familiar with during earlier labs. For software testing, we employed a C testing framework and applied the same strategies and techniques used throughout the course.

Single-Core System Hardware Testing:

The first step in our testing process was to verify that the multiplier, processor, and cache implementations from previous labs passed all tests and functioned correctly. Once we confirmed the correctness of these components, we proceeded to test the single-core system. In Lab 2, we performed extensive unit testing of our processor for each instruction. Building on that foundation, we reused assembly tests from Lab 2 to test our single-core system and identify potential bugs. Finally, we added mixed instruction tests (which we didn't do for lab 2), and they helped stress test the single core and ensure proper functionality.

To thoroughly test the partition function, we designed a variety of scenarios. Starting with a basic partition of a whole array, we tested an unsorted array like {4, 3, 6, 5} to ensure the pivot is correctly placed, with all elements smaller or equal to the pivot on the left and larger ones on the right. We also tested arrays with repeated values, such as {7, 7, 7, 7, 7}, to verify that the pivot remains at the correct index without unnecessary rearrangements. For arrays with very large values, such as {1000000, 500000, 1000000, 2000000}, we ensured the partitioning handles large integers without errors. A single-element array {7} was used to confirm the function properly handles minimal input, where the pivot remains unchanged. To validate functionality with out-of-order arrays, we tested scenarios like {8, 1, 4, 6, 3} to ensure the pivot is placed correctly. We also included arrays with negative numbers, such as {-3, 2, -5, 7, -1}, to ensure the function handles mixed positive and negative values appropriately. For sorted arrays, such as {1, 2, 3, 4, 5}, the pivot should naturally be placed at the end without rearrangement. Conversely, with reverse-sorted arrays, such as {5, 4, 3, 2, 1}, the pivot should move to the first index, confirming correct handling of reversed input. We also tested partitioning different sub-intervals within arrays, like {8, 3, 7, 6, 2, 5} over the subrange {3, 7, 6}, to ensure the function handles subsets accurately. Finally, a sequence of partitions was tested by recursively applying the function on arrays like {9, 7, 8, 6, 5, 4, 3}, simulating its behavior in a quicksort context.

Single-Core System Software Testing:

To validate the sorting implementation, we constructed test cases covering diverse input scenarios. A basic unsorted array, such as {4, 3, 6, 5}, confirmed the function correctly sorts a small dataset. Testing with reverse-sorted arrays, such as {10, 8, 6, 4, 2}, ensured proper handling of worst-case input. Similarly, an already sorted array, like {1, 2, 3, 4, 5}, verified the function doesn't unnecessarily rearrange elements. We also used a single-element array {7} to confirm the algorithm gracefully handles trivial cases. Arrays with repeated elements, such as {5, 3, 3, 7, 7, 2}, tested the function's robustness with duplicates. Additionally, we validated handling of negative numbers, such as {-3, 2, -5, 7, -1}, ensuring mixed positive and negative values are sorted correctly. Testing uniform arrays, like {5, 5, 5, 5, 5}, ensured no unnecessary operations on identical elements. For larger datasets, we used random arrays with 100 elements to verify performance and correctness. Finally, we simulated multiple recursive calls using complex input sequences, ensuring the function's behavior scales as expected.

4. Optimizations

In terms of hardware optimizations, the main improvement is the implementation of the shortest path algorithm in the ring network router's route unit, as outlined in the alternative design section. The shortest path algorithm provides more efficient routing compared to the clockwise routing approach. For instance, if the current router is router 2 and the destination router is router 1, the shortest path algorithm would route one unit to the left, while the clockwise algorithm would route three units clockwise to reach router 1. In the worst-case scenario, the shortest path algorithm would only require moving two units, as opposed to three units in the clockwise algorithm, thus reducing latency and improving overall network efficiency.

For software optimizations related to the sorting algorithms developed for our single-core and multi-core processors, we chose to implement hybrid sorting algorithms (quicksort combined with insertion sort) rather than relying on a single algorithm. This hybrid approach leverages the strengths of both algorithms: quicksort efficiently handles larger datasets with its divide-and-conquer strategy, while insertion sort performs better on small subarrays, reducing overhead and improving performance for smaller datasets. We found that using insertion sort for arrays of size less than or equal to 10 and quicksort for arrays larger than size 10 provided the most optimal speed (number of cycles) improvement. This combination enhances overall performance across a wide range of input sizes. In the multi-threaded software, merging is added on top of the hybrid quicksort algorithm, with each of the four threads/cores sorting a quarter of the data array. To optimize the merging process, merging is initially delegated to two threads (core 0 and core 1), and the final sorted array is merged on core 0, maximizing parallelism and reducing the merging time.

5. Evaluation

To evaluate the performance of our baseline single-core and alternative multi-core processor designs, we conducted a series of five experiments using single-threaded and multi-threading microbenchmark programs: *bsearch*, *cmult*, *mfilt*, *sort* (custom developed), and *vvadd*. Each of these programs was run on five different processor configurations: single-thread single-core, single-thread multi-core, multi-thread single-core, multi-thread multi-core with 1 worker, and multi-thread multi-core with 4 workers. There are 25 experiments in total across all microbenchmarks and processor configurations. The results of these experiments, including CPI and number of cycles, are used to quantify the software and hardware overheads of the multi-core system and determine whether the alternative design (multi-core) achieves a speedup over the baseline. An overview of each microbenchmark program is as follows:

- **bsearch:** Computes a binary search algorithm on a dictionary. In the single-thread implementation, the whole dictionary is searched through using one thread. In the multi-thread implementation, the keys are partitioned into each thread and searched in parallel with each other.
- **cmult:** Computes the product of two complex numbers. In the single-thread implementation, only one core goes through the whole array while in the multi-thread implementation, the input array is split into blocks and processed on each thread separately. The final result is merged at the end.
- **mfilt:** Computes the median filter result on a 2 dimensional array input. The single-thread implementation involves processing an entire input matrix using one core, while the multi-thread implementation splits this matrix into blocks for parallel processing across several threads. Each thread handles a separate section of the 2D input array and considers edge cases when merging.
- **sort: (custom)** Computes a hybrid quicksort algorithm on a data array of integers (previously mentioned in optimization section). The single thread implementation utilizes a mix of quicksort and insertion sort depending on the input array size. The multi-thread implementation builds upon this by using this hybrid quicksort algorithm on a split up input array on each thread and then merging at the end.
- **vvadd:** Computes a vector-vector addition on two arrays. The single-thread implementation goes through the entire array iteratively, while the multi-thread implementation splits the array into four blocks and processes vector addition on each thread concurrently and finally merges at the end.

Processor Configuration 1: Single-Thread Single-Core

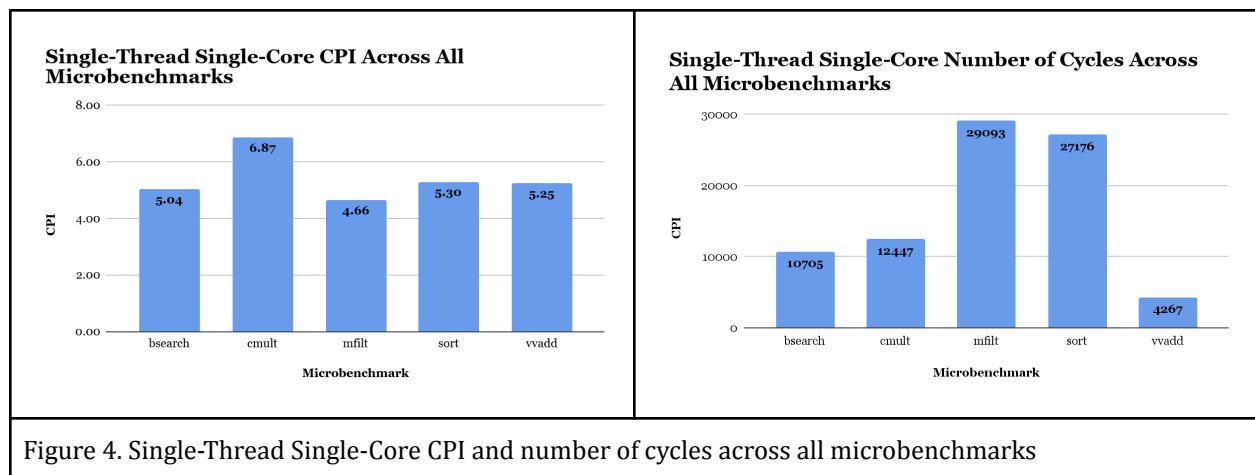
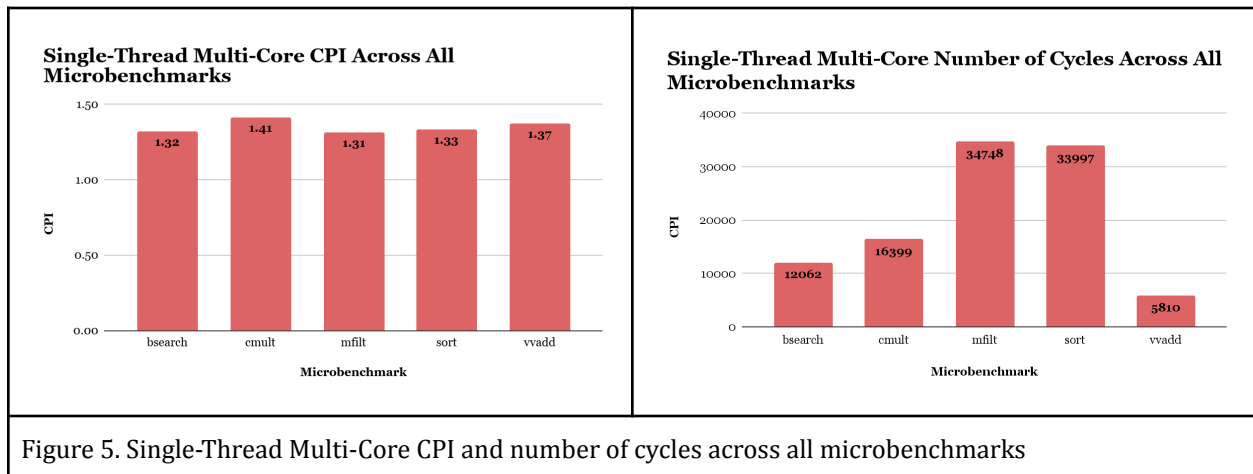


Figure 4. Single-Thread Single-Core CPI and number of cycles across all microbenchmarks

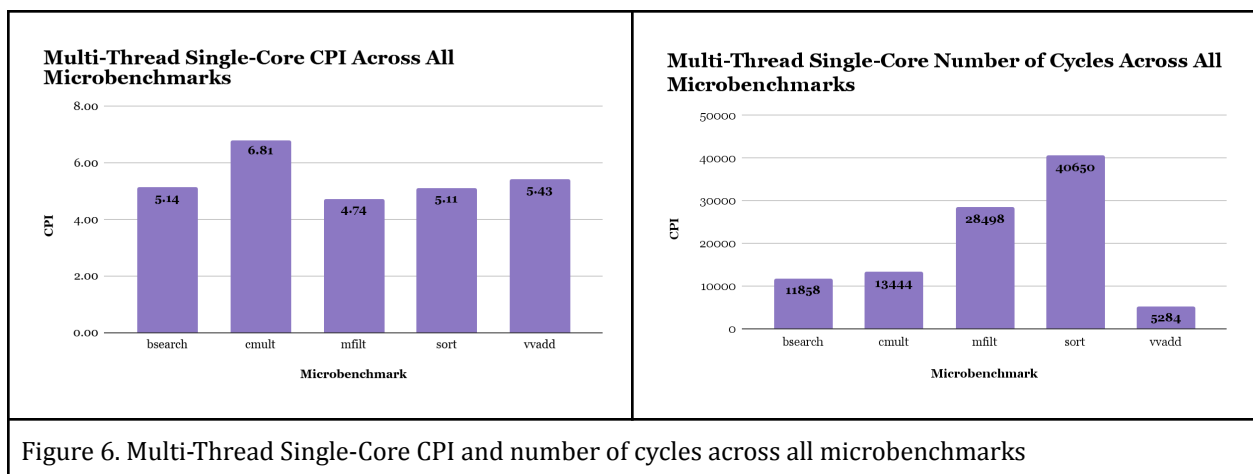
Processor configuration 1, the single-thread single-core scenario, provides a baseline performance metric for all other processor configurations to compare to. As seen above in Figure 4, the CPI of the single-thread single-core processor is fairly high, hovering around the 4-6ish range while execution time (number of cycles) has high variability depending on the microbenchmark. In terms of CPI, mfilt performs the best as it has the lowest CPI of 4.66 while cmult performs the worst with the highest CPI of 6.87. This could potentially be due to the nature of complex numbers and dealing with larger number values when on the register level. In terms of execution time, vvadd has the fastest execution time at 4267 cycles while mfilt has the longest execution time at 29093 cycles. Interestingly, mfilt has the best CPI while having the worst execution time, indicating that this microbenchmark is likely high in instruction count and software dependent on performance.

Processor Configuration 2: Single-Thread Multi-Core



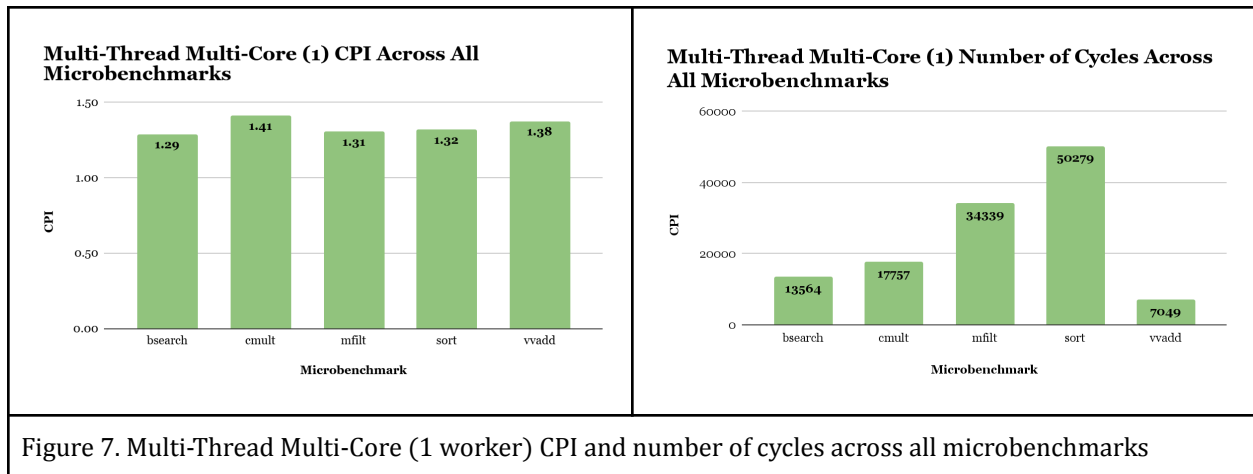
In processor configuration 2, all single-thread microbenchmark programs are run on the multi-core processor rather than the single-core processor. Figure 5 above illustrates the CPI and execution time of all microbenchmark programs on this configuration. The CPI is fairly consistent across all programs, hovering around the 1.3-1.4 number while the execution time is largely variable, similar to the baseline processor configuration 1. The trend continues with cmult having the worst CPI and mfil having the best CPI (although the margins are much smaller this time). Execution time also follows the same trend with mfil having the worst execution time and vvadd having the best execution time.

Processor Configuration 3: Multi-Thread Single-Core



The next processor configuration moves onto multi-thread microbenchmark programs but running on a single-core processor. Figure 6 above illustrates the CPI and execution time of this processor across all microbenchmark programs. Note that these multi-threaded programs were not meant to run on a single core processor but rather software translated in order for it to work. Looking at the numbers, the CPI of this processor performs fairly bad and varies from ranges of 4.74 to 6.81. This is somewhat similar to the baseline processor configuration, albeit slightly better. The same trend for CPI from the other processors continues, where mfiltr has the best CPI and cmult has the worst CPI. Interestingly however, sort has the worst execution time now instead of mfiltr, indicating that the software implementation of the multi-threaded sorting algorithm is not as ideal as it could have been.

Processor Configuration 4: Multi-Thread Multi-Core (1 Worker)



In processor configuration 4, multi-threaded programs are tested on a multi-core processor, however with only 1 worker instead of the default 4, so that only 1 core is active at a time. Figure 7 above illustrates the numerical statistical performance of this processor. The CPI of this goes down to the 1.3-1.4 range, similar to processor configuration 2. Overall, the CPI of the multi-core processors are much lower than single-core processors. The worst CPI occurs with cmult while the best CPI occurs with mfiltr, similar to other processors. Like processor 3, the worst execution time is with sort and the best execution time is with vvadd.

Processor Configuration 5: Multi-Thread Multi-Core (4 Workers)

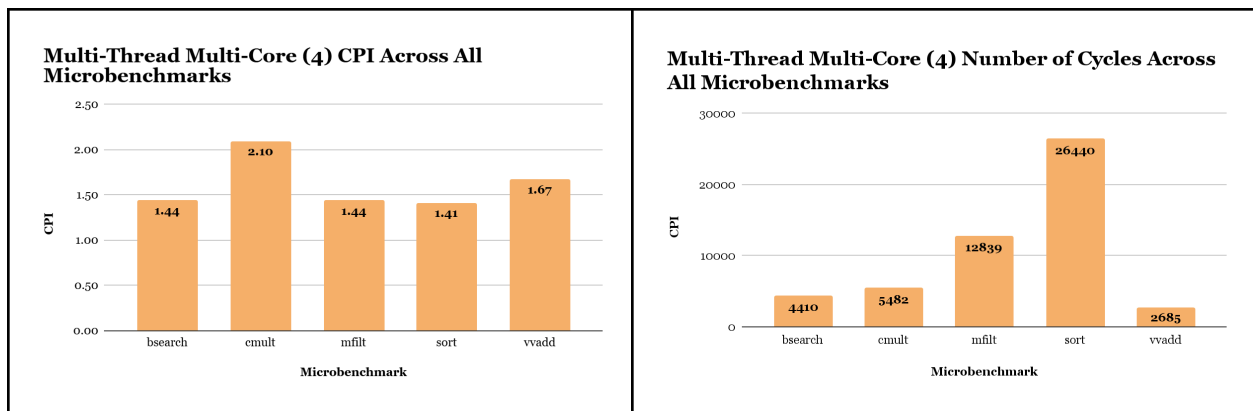


Figure 8. Multi-Thread Multi-Core (4 workers) CPI and number of cycles across all microbenchmarks

Lastly, processor configuration 5 tests multi-threaded programs on the entire multi-core processors with all 4 workers utilized, so that all 4 cores can work in parallel. Statistical number results are plotted above in Figure 8. The CPI of this is slightly worse than processor configuration 4 with just 1 worker, indicating that adding more cores in general stresses the hardware performance of the processor as multiple memory accesses and data cache utilizations are challenging, especially when synchronization is necessary. Likewise with other multi-threaded processor configurations the same trend holds: mflt has the best CPI, cmult has the worst CPI, vvadd has the best execution time, and sort has the worst execution time.

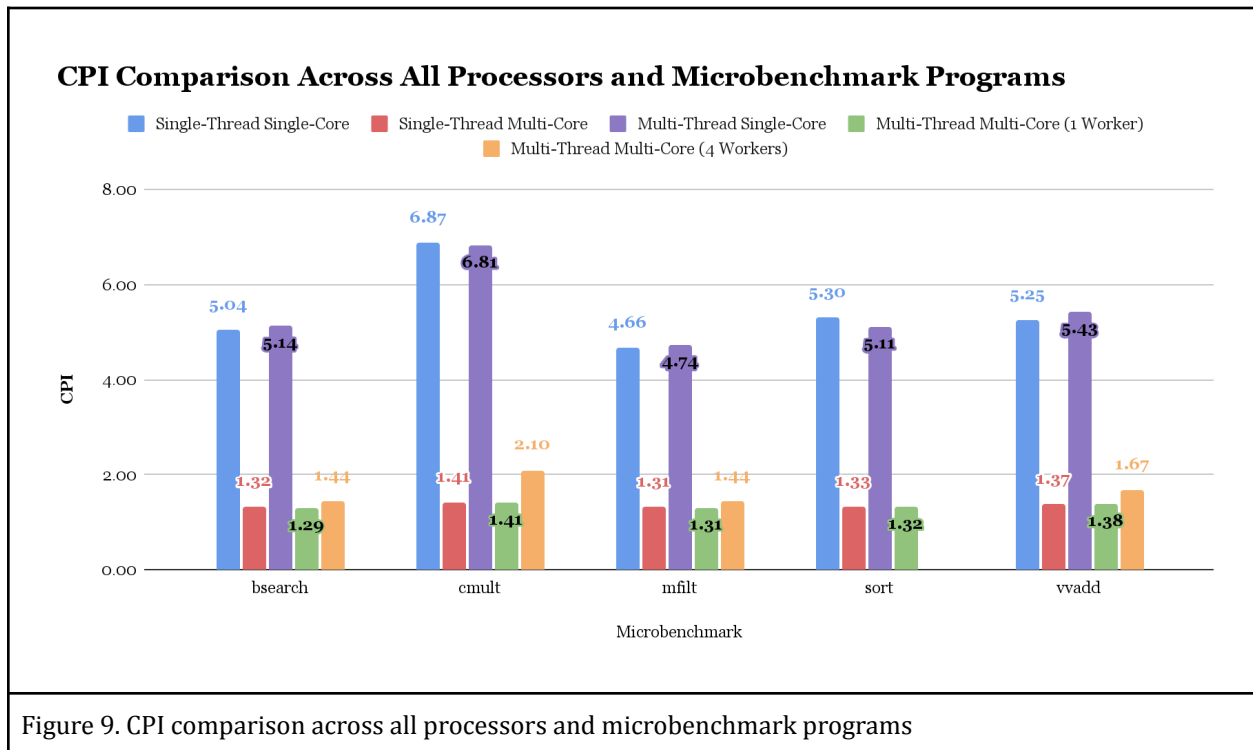
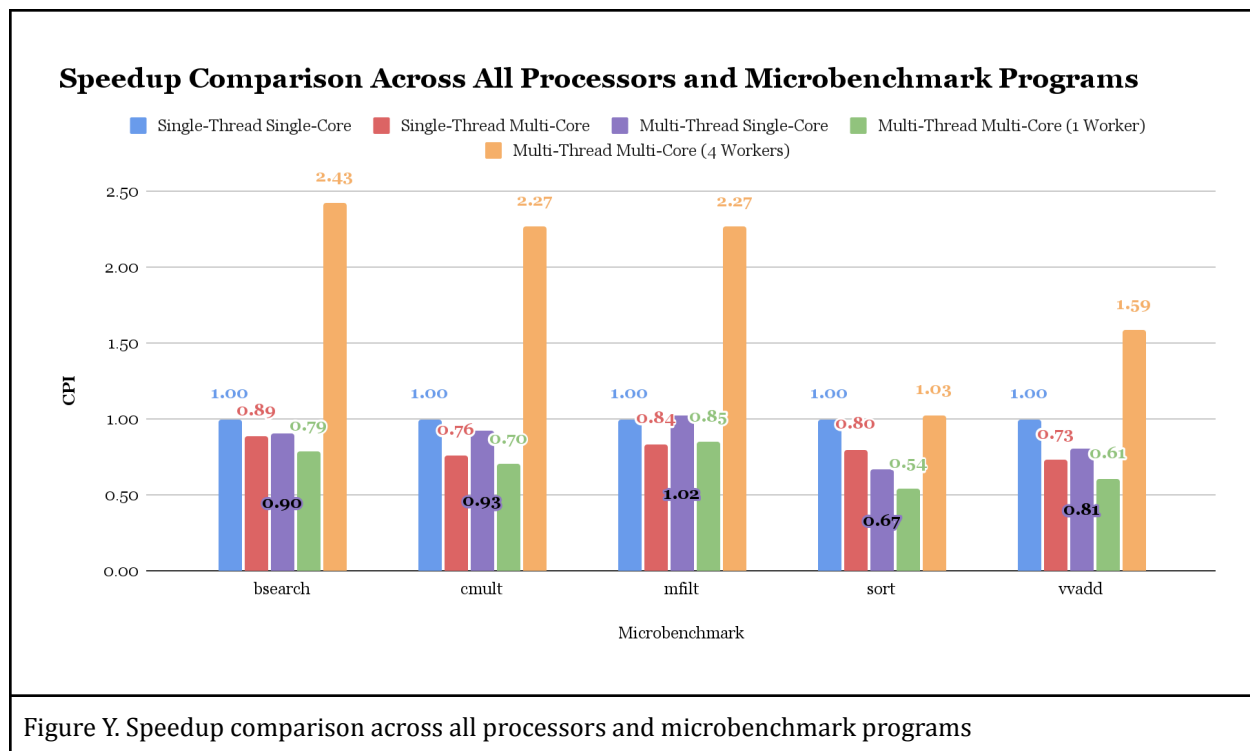


Figure 9. CPI comparison across all processors and microbenchmark programs

Putting all this information together, we can compare the CPI across all processor configurations and microbenchmark programs. From figure 9, we can tell that as a general trend, single-core processors have a worse CPI that hovers around the 4-7 range while multi-core processors have a better CPI that hovers around the 1-2 range. The baseline design with a single-thread single-core processor has the worst CPI overall while the multi-thread multi-core processor with 1 worker has the best CPI overall.



Comparing across all execution times, the baseline single-thread single-core processor has a fairly decent execution time that outperforms 3 of the 4 other processors.

6. Conclusion

In summary, this lab examined the performance trade-offs between single-core and multi-core systems using both single-threaded and multi-threaded sorting microbenchmarks. Through five distinct experiments, we analyzed the baseline design performance, evaluated hardware and software overheads, and assessed the efficiency of the alternative multi-core design. The results highlighted significant differences between the two approaches, with the multi-core design achieving speedups of up to 2.42x in bsearch-eval and 2.27x in mfilt-eval compared to the single-threaded ubmark baseline. These findings demonstrated that multi-core systems, when effectively utilizing all cores with multi-threaded software, can deliver substantial speedups and reduced execution times. However, the analysis also revealed notable overheads from synchronization and thread management, reflected in increased CPI values for some benchmarks. Overall, this lab emphasized the importance of balancing hardware complexity with software optimization to create scalable and efficient multi-core systems. It reinforced key concepts in computer architecture, such as modularity, incremental design, and the synergy between hardware and software, equipping us to address future challenges in system-level performance optimization.

7. Work Distribution

We split our work fairly evenly as a group. Git version control and VSCode live share served as two important collaboration tools during this lab. On the baseline design, Edwin worked baseline and most of the testing while Laurence worked on the multicore hardware and software. For testing, Edwin wrote pytest test cases while Laurence wrote evaluations. On the alternative design, we both worked simultaneously together on implementing the design in person using VSCode live share. In the report, Edwin worked on the introduction and alternative design while Laurence worked on the evaluation and conclusion. Both members worked on the testing section of the report.

